

attempting to do. The remainder of this chapter describes Mach, BSD, and the I/O Kit and outlines the functionality that is provided by those components.

The process & the operating system OS . These two abstractions lets us switch the state of the process between multiple tasks. The process clearly defines the status of an executing between programs while the OS provides the mechanism for switching execution between the processes.

Operating System Basics

- ☐ Bridges between user application/tasks & system resources
 - ☐ The OS manages the system resources & make them available to the user application/task.
- computing system is collection of different I/O sub-system working & storage memory.

Primary FunctionOf OS

- ☐ Make the system convenient to use
- ☐ Organizes & manage the system resources efficiently & correctly

The Kernel

The kernel is core of the OS & is responsible for managing the system & the communication among the hardware & the system services. Kernel acts as the abstraction layer between system resources & user application. Kernel contains a set of system libraries of services. For general purpose OS the kernel contains different services for handling the management.

1. Process Management: Managing the

- ☐ process/tasks
- ☐ Setup the memory space for process
- ☐ Load program/code into space(memory)
- ☐ Scheduling & managing the execution of the process
- ☐ Setting up & managing the PCB

- ☐ Inter process communication & system synchronization process termination & deletion

2. Primary Memory Management

- ☐ It is volatile memory
- ☐ The MMU of kernel is responsible for
 - Keeping track of which part of memory area is correctly used by which process

Embedded Computing System 10CS72

Department Of CSE,ACE Page 45

- Allocating & deal locating the memory spaces

3. File System Management

- ☐ File is a collection of related information
- ☐ A file could be a program, text files, image file, word documents, audio/video files etc.
- ☐ The file system management service of kernel is responsible for
 - The creation & deletion of files
 - Creation , deletion & alteration directly
 - Saving the file in secondary storage memory
 - Providing automatic allocation of spaces
 - Providing a flexible naming convention for the files

4. I/O System(Device) Management

- ☐ Kernel is responsible for routing the I/O request coming from different user applications
- ☐ Direct access of I/O devices are not allowed , we can access the I/O

devices through the API imposed by kernel

- ☐ Dynamically update the available devices
- ☐ Device manager of the kernel is responsible for handling I/O device related operations
- ☐ The kernel talks to the I/O device through the device driver , this is responsible for
 - Loading & unloading of device drivers
 - Exchanging the information's

5. Secondary Storage Management

- ☐ The secondary storage management deals with secondary storage memory device , if any connected to the system
- ☐ The main memory is volatile
- ☐ The secondary storage management services of kernel deals with
 - Disk storage allocation
 - Disk scheduling
 - Free disk space management

What is RTOS?

1) A real time operating system (RTOS) is an operating system that guarantees a certain **capability** within a **specified time constraint**.

2) An OS is a system program that provides an **interface** between

Application programs and the computer system (**hardware**)

3) The applications where dependability that a certain task will finish before a particular **deadline** is just as obtaining the **correct results**.

4) Besides meeting deadlines RTOS must also be able to respond **predictably** to unpredictable **events** and process **multiple events** concurrently.

A **Real-Time Operating System** (RTOS) is a computing environment that reacts to input within a specific time period. A real-time deadline can be so small that system reaction appears instantaneous. The term real time computing has also been used, however, to describe "slow real-time" output that has a longer, but fixed, time limit.

Learning the difference between real-time and standard operating systems is as easy as imagining yourself in a computer game. Each of the actions you take in the game is like a program running in that environment. A game that has a real-time operating system for its environment can feel like an extension of your body because you can count on a specific "lag time:" the time between your request for action and the computer's noticeable execution of your request. A standard operating system, however, may feel disjointed because the lag time is unreliable. To achieve time reliability, real-time programs and their operating system environment must prioritize deadline actualization before anything else. In the gaming example, this might result in dropped frames or lower visual quality when reaction time and visual effects conflict.

Real-Time Kernel

The heart of a real-time OS (and the heart of every OS, for that matter) is the **kernel**. A kernel is the central core of an operating system, and it takes care of all the OS jobs:

1. Booting
2. Task Scheduling
3. Standard Function Libraries

In an embedded system, frequently the kernel will boot the system, initialize the ports and the global data items. Then, it will start the scheduler and instantiate any hardware timers that need to be started. After all that, the Kernel basically gets dumped out of memory (except for the library functions, if any), and the scheduler will start running the child tasks. Standard function libraries: In an embedded system, there is rarely enough

memory (if any) to maintain a large function library. If functions are going to be included, they must be small, and important.

Basic Kernel Services

Kernel which is a major part of an operating system that provides the most basic services to application software running on a processor. The "kernel" of a real-time operating system ("RTOS") provides an "abstraction layer" that hides from application software the hardware details of the processor (or set of processors) upon which the application software will run.

Scheduling

In typical designs, a task has three states:

1. Running (executing on the CPU);
2. Ready (ready to be executed);
3. Blocked (waiting for an event, I/O for example).

Most tasks are blocked or ready most of the time because generally only one task can run at a time per CPU. The number of items in the ready queue can vary greatly, depending on the number of tasks the system needs to perform and the type of scheduler that the system uses. On simpler non-pre-emptive but still multitasking systems, a task has to give up its time on the CPU to other tasks, which can cause the ready queue to have a greater number of overall tasks in the ready to be executed state (resource starvation). Usually the data structure of the ready list in the scheduler is designed to minimize the worst-case length of time spent in the scheduler's critical section, during which pre-emption is inhibited, and, in some cases, all interrupts are disabled. But the choice of data structure depends also on the maximum number of tasks that can be on the ready list.

If there are never more than a few tasks on the ready list, then a doubly linked list of ready tasks is likely optimal. If the ready list usually contains only a few tasks but occasionally contains more, then the list should be sorted by priority. That way, finding the highest priority task to run does not require iterating through the entire list. Inserting a task then requires

walking the ready list until reaching either the end of the list, or a task of lower priority than that of the task being inserted.

Care must be taken not to inhibit pre-emption during this search. Longer critical sections should be divided into small pieces. If an interrupt occurs that makes a high priority task ready during the insertion of a low priority task, that high priority task can be inserted and run immediately before the low priority task is inserted.

The critical response time, sometimes called the fly back time, is the time it takes to queue a new ready task and restore the state of the highest priority task to running. In a well-designed RTOS, readying a new task will take 3 to 20 instructions per ready-queue entry, and

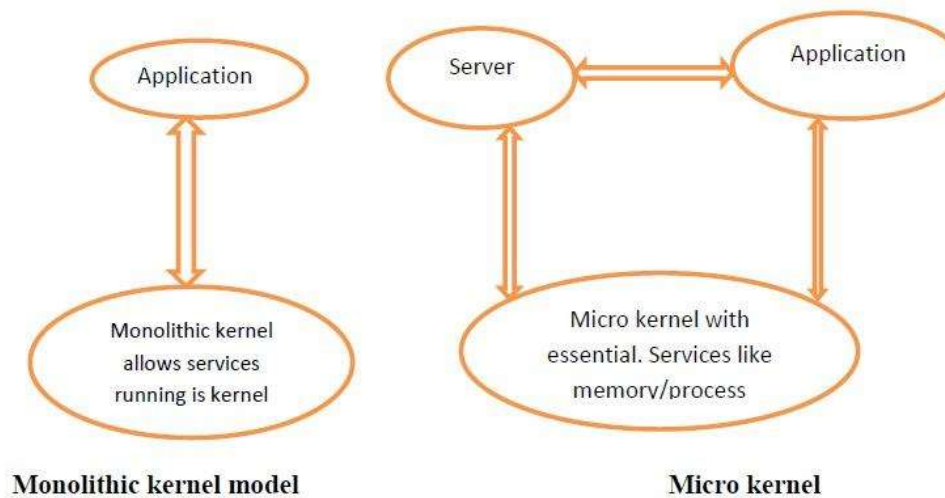
restoration of the highest-priority ready task will take 5 to 30 instructions.

In more advanced systems, real-time tasks share computing resources with many non-real-time tasks, and the ready list can be arbitrarily long. In such systems, a scheduler ready list implemented as a linked list would be inadequate.

Types of Kernel:

Based on kernel design, kernel can be classified into Monolithic & Micro

- ☐ In Monolithic kernel architecture all the kernel service run in the kernel space – means all kernel module run with same memory space under single kernel thread
- ☐ The drawback of Monolithic kernel is that any error or failure in any of the kernel module i.e. leads to the crashing of entire kernel application. Ex : LINUX,SOLARIS,MS-DOS



Micro kernel: design incorporates only the essential set of OS services into the kernel. The rest of the OS services are implanted in programs known as “servers” which runs in user space.

The essential services of the Micro kernel are

- ☐ Memory management
- ☐ Process management Timer
- ☐ system
- ☐ Interrupt handler

Ex. OS , MACH , ONX , MINIX3

Benefits of Micro kernel

1. Robust
2. Configurability

Types of Operating System

Depending on the type of kernel & kernel services purpose & type of in computing system OS are classified into two types

1. General Purpose Operating System[GPOS]
2. Real Time Operating System[RTOS]

1. General Purpose Operating System[GPOS]

The OS which are deployed in general computing systems are referred as general purpose OS.

The kernel of such OS is more generalized & it contains all kinds of services required for executing generic applications

- ☐ GPOS are non-deterministic in behaviour

Ex: PC/Desktop system, windows XP & MS-DOS

2. Real Time Operating System [RTOS]

There is no universal definition available for the term RTOS.

- ☐ A RTOS is an OS intended to serve real-time application requests. It must be able to process the data as it comes in typically without buffering delays
- ☐ Processing time requirements are measured in tenth of seconds. Hard

RTOS has less jitter than soft RTOS.

[Jitter: Variation in time between packet arriving with time congestion & time.]

- ☐ Real-time implies deterministic timing behaviour – means the OS

services consumes only known & expected amount of time regardless the no of services.

- RTOS decides which application should run in which order & how much time needs to be allocated for each applications.

Ex: Windows CE, UNIX, VxWorks, Micros/OS-II Real Time

Kernel:

The RTOS is referred to as Real Time kernel in complement to conventional OS kernel. The kernel is highly specialized & it contains only the minimal set of services required for running the user application/

Tasks.

The Basic Functions of Real Time Kernel:

1. Task/Process Management
2. Task/Process Scheduling
3. Task/Process Synchronization
4. Error/Exception Handling
5. Memory Management
6. Interrupt Handling
7. Time Management

Task/Process Management: Deals with setting up the memory space for the tasks, loading the tasks code into memory space, allocating system resources, setting up a Task Control Block [TCB] for the task & task/process termination/deletion.

TCB: Task Control Block is used for holding the information corresponding to a task. The TCB contains the following set of information

Task ID: Task identification number

- Task State: The current state of the task

- ☐ Task Type: Indicates what is the type of this task, the task can be hard real time or soft real time or background task
- ☐ Task Priority: Ex task priority=1 for task with priority =1
- ☐ Task Context Pointer: context pointer, pointer for saving context
- ☐ Task Memory Pointer: Pointers to the code memory, data memory & stack memory for the task
- ☐ Task System Resource Pointers: Pointer to system resource used by the task
- ☐ Task Pointers: Pointer to other TCB's
- ☐ Other Parameters: Other relevant task parameters

The TCB parameters vary across different kernels based on the task management Implementation

- ☐ Creates TCB for a task on creating task
- ☐ Delete/Remove the TCB when the task is terminated or deleted
- ☐ Reads the TCB to get the state of a task
- ☐ Updates the TCB with updated parameters on need basics
- ☐ Modify the TCB to change the priority of task dynamically

Task/Process Scheduling: Sharing the CPU among various tasks/process. A kernel applications called scheduler, handles the task scheduling. Scheduler is nothing but an algorithm implemented all to , which performs the efficient & optimal scheduling of task to provide a deterministic behaviour.

Task/Process Synchronization: deals with system concurrent accessing a resource which is shared access multiple tasks & communication between various tasks.

Error/Exception Handling: Deals with registering & handling the errors occurred during the execution of tasks. Ex: Insufficient memory, time outs, dead locks, dead line missing, bus error, divide by zero, unknown instruction execution. Errors & exceptions can happen at two levels of services * Kernel Level Service * At Task Level

Memory Management: RTOS makes use of 'Block Based Memory' allocation techniques instead of the usual dynamic memory allocation technique used by GPOS. RTOS kernel uses blocks of fixed size dynamic memory & block is allocated for a task on a need of basis. A few RTOS kernel implements Virtual Memory concepts avoid the garbage collection overhead.

Interrupt Handler: Deals with the handling of several of interrupts. Interrupts provide Real Time behaviour embedded system design process

to systems. Interrupts inform the processor that an external device or an associated task required immediate attention of the CPU. Interrupts can

be either Synchronous Asynchronous. Interrupts which occurs in synch with the currently executing task is known as synchronous interrupts. Usually the system interrupts fall under the synchronous category Ex

Divide by zero, memory segmentation error. A synchronous interrupts are those which occurs at any point of execution of any task & are not in sync with currently executing tasks.

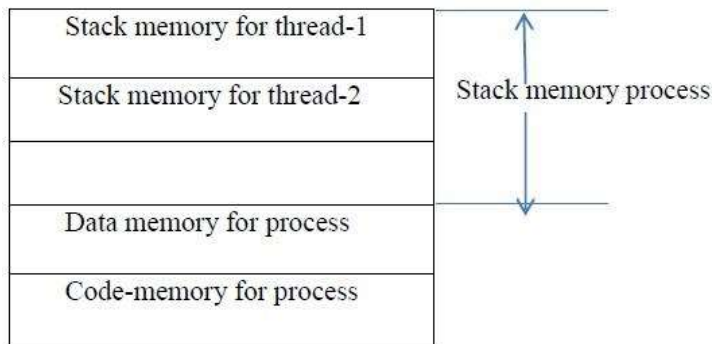
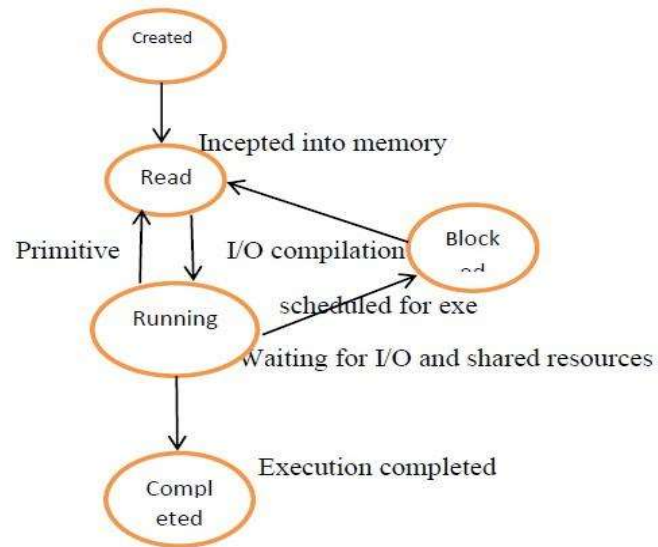
Task, Process & Threads

Task: Refers to something that needs to be done. The tasks refers can be one assigned by manages or the one assigned by only one of the processor family needs. In addition we have an order of priority & scheduling timeline for executing these tasks. In OS context, a task is defined as the program in execution & related information maintained by the OS. Task is also known as "JOB" in the OS context.

Process: A process is a program as part of the program, in execution process is also known as instance of the program in execution, multiple instance of the same program can execute simultaneously. Process requires various system resources like CPU for executing the

process, memory for storing the code corresponding to the process, I/O devices for in function exchange etc.

Process States & State Transition



Memory organization of a process and its associated threads Run: A task enters this state as it starts executing on the process Ready state of those tasks that are ready to execute but

cannot be executed because the processor is assigned to another task Blocked a task enters this state when it executes synchronization primitive to wait for an event

Ex:-a wait primitive on a semaphore this case the task is inserted in a queue associated with the semaphore.

Created (idle) a periodic job enters this state when it completes its execution and has to wait for the beginning of the next period **Process management:**

Process management deals with the creating a process, setting up the

memory space for the process, loading the process code with the memory space, allocating the system resources. Setting up a process control block (PCB) for the process and process termination/detection. **Task scheduling:**

Multitasking involves the execution switching among the different tasks. There should be some mechanism in place to share the CPU among the

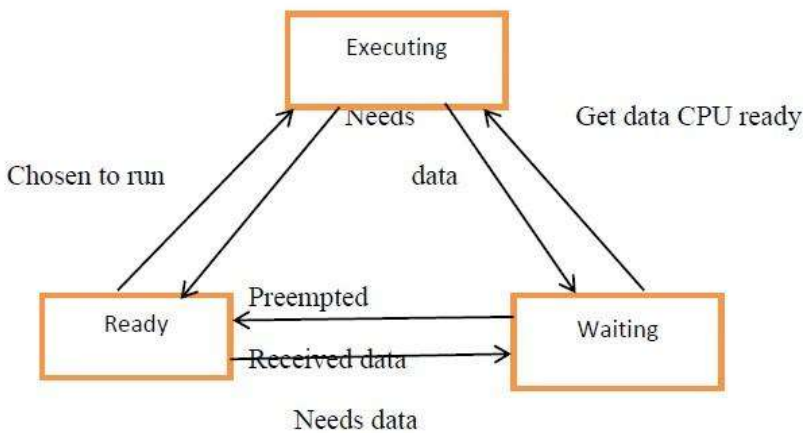
different tasks and to decide which process/task is to be executed at a given point of time.

Determining which task/process is to be executed at a given point of time is known as task/process scheduling. Task

scheduling from the basis of multitasking Scheduling policies from the guidelines for determining which task is to be executed when.

The work of choosing the order of running process is known as scheduling. A scheduling policy defines how processes are selected for

promotion from the ready state to the running state. The process scheduling decision may take place when a process switches its states Ready state from Running state Blocked/wait state from Running state Ready state from Blocked/wait state Completed (executed) state



Scenario 1 is primitive: a process switches to ready state from the running state

Scenario 2 is scheduling under can be either pre-emptive or non-pre-emptive. When a high priority process in the blocked/wait state complete its I/O and switches to ready state. The scheduler picks it for execution if the scheduling policy used is priority based pre-emptive.

Semaphores

A **semaphore**, in its most basic form, is a protected integer variable that can facilitate and restrict access to shared resources in a multi-processing environment. The two most common kinds of semaphores are **counting semaphores** and **binary semaphores**. Counting semaphores represent multiple resources, while binary semaphores, as the name implies, represents two possible states (generally 0 or 1; locked or unlocked). Semaphores were invented by the late Edsger Dijkstra.

Semaphores can be looked at as a representation of a limited number of resources, like seating capacity at a restaurant. If a restaurant has a capacity of 50 people and nobody is there, the semaphore would be initialized to 50. As each person arrives at the restaurant, they cause the seating capacity to decrease, so the semaphore in turn is decremented. When the maximum capacity is reached, the semaphore will be at zero, and nobody else will be able to enter the restaurant. Instead the hopeful restaurant goers must wait until someone is done with the resource, or in this analogy, done eating. When a patron leaves, the semaphore is incremented and the resource becomes available again.

A semaphore can only be accessed using the following operations: **wait()** and **signal()**. **wait()** is called when a process wants access to a resource. This would be equivalent to the arriving customer trying to get an open table. If there is an open table, or the semaphore is greater than

zero, then he can take that resource and sit at the table. If there is no open table and the semaphore is zero, that process must wait until it becomes available. **signal()** is called when a process is done using a resource, or when the patron is finished with his meal. The following is an implementation of this **counting semaphore** (where the value can be greater than 1):

```
wait(Semaphore s){
    while (s==0); /* wait until s>0 */
    s=s-1;
}

signal(Semaphore s){
    s=s+1;
}

Init(Semaphore s , Int v){
    s=v;
}
```

Historically, **wait()** was called **P** (for Dutch “Proberen” meaning *to try*) and **signal()** was called **V** (for Dutch “Verhogen” meaning *to increment*). The standard Java library instead uses the name "acquire" for **P** and "release" for **V**.

No other process can access the semaphore when P or V are executing. This is implemented with **atomic** hardware and code. An atomic operation is indivisible, that is, it can be considered to execute as a unit.

If there is only one count of a resource, a **binary semaphore** is used which can only have the values of 0 or 1. They are often used as **mutex locks**. Here is an implementation of mutual-exclusion using binary semaphores:

```
do
{
    wait(s);
    // critical section
    signal(s);
    // remainder section
} while(1);
```